

ReadWithPauline
API MVP
(v4.1.4)

Context & Objectives

Business needs:

- Online bookstore platform
- Customers must browse, search, and purchase books online

Technical scope:

- Frontend : Web application (consuming the API)
- Back end : REST API

MVP Functional Scope

4 core features:

1. Books – list, search, details, availability
2. Customers – accounts & address rules
3. Orders – purchases, payments, stock updates
4. Notifications – back-in-stock alerts

Scope : France and Belgium (MVP)

=> **4 different endpoints** for business requirements (cf [SwaggerHub](#))

Architecture Overview

Split in 4 layers:

1. Frontend web app (UI)
 2. API Gateway + Auth
 3. REST API backend
 4. MySQL DB + background jobs (notifications and emails)
- Flow: Client → API Gateway → Backend → DB → Email job (every 5 min)
 - All communication via HTTPS + JSON.

Authentication & Roles

Current: **API Key** (MVP) | Next: JWT

3 permission levels :

1. Customer (cus_): auto, expires after 90 days
 2. Admin (adm_): manually manage
 3. Super Admin (sup_): full access
- Every request (except registration and login) must include Authorization: ApiKey <key>
 - All calls HTTPS only (403 if HTTP)
 - GDPR: actions logged 90 days
 - 2 endpoints for authentication

Data Model & Field mapping

Every API field is mapped to the physical database model.

Examples with the main table :

BOOKS

- books_id (PK, AUTO_INCREMENT)
- ISBN, EAN (UNIQUE)
- stock_quantity, price

CUSTOMERS

- customer_id (UUID)
- email (UNIQUE)
- password_hash, address

ORDERS

- order_id (INT, internal)
- order_code (public: ORD-YYMMDD-XXXX)
- status, total_amount

Business Rules & Conversions

Customers:

- Auto-capitalize names
- Phone to E.164
- Passwords are never stored and hashed (bcrypt)

Addresses:

- FR & BE only (ISO codes on 2 digits)
- Postal code validated per country

Orders:

- OrderCode: ORD-YYMMDD-XXXX (human-readable)
- Snapshot shipping address

Notifications:

- Default = pending

All the fields with a “Date” type are converted to ISO 8601 timestamp

Error Handling

Standard JSON error:

```
{  
  "timestamp": "2025-10-30T14:30:00Z",  
  "status": 400,  
  "error": "Bad Request",  
  "code": "BOOK_OUT_OF_STOCK",  
  "message": "Only 2 available",  
  "path": "/api/orders",  
  "details": {"requested": 5}  
}
```

45+ custom error codes:

- Auth errors
- Validation (books, customers, orders, notifications) => functional errors are mapped to specific HTTP codes

Predictable errors = easier integration.

Performance & Reliability Targets

Response time targets:

- GET <200ms
- POST/PUT <1000ms
- DELETE <500ms
- Rate limit: max 200 req/min (too many request → 429)

Consistency:

- Valid key + valid JSON response
- HTTPS is mandatory
- UTC timestamps
- All required fields must be present
- All FK validated.

Test Coverage

Positive + negative tests per domain :

Examples :

- **Books:**
 - Get all books → 200 OK, returns array.
 - Get book by ID → 200 OK with full details.
 - Book not found → 404 BOOK_NOT_FOUND.
 - Duplicate ISBN → 409 Conflict
- **Customers:**
 - Register a new customer → 201 Created. Generates UUID, hashes password.
 - Weak password → 400 CUSTOMER_WEAK_PASSWORD.
 - Wrong country → 400 CUSTOMER_INVALID_COUNTRY (we only allow FR and BE).
 - Email already used → 409 CUSTOMER_EMAIL_EXISTS.
- **Orders:**
 - Place valid order → 201, status = processing, stock decremented.
 - Try to buy out-of-stock → 409 BOOK_OUT_OF_STOCK.
 - Payment failed → 422 ORDER_PAYMENT_FAILED.
 - Cancel order:
 - Allowed if status=processing → 200 OK and stock restored.
 - Not allowed if status=shipped → 409 ORDER_CANNOT_CANCEL.

Database & Indexes & Relationships

5 core tables: books, customers, orders, order_items, notifications

Relations (1:N validated by FK):

- Customer → Orders
- Order → Items
- Book → Items
- Customer → Notifications
- Book → Notifications

Indexes:

- Books: category, author, stock
- Customers: email
- Orders: status, date
- Notifications: book, customer

Optimized for performance.

User Stories / Roadmap

-  US-01 Register
-  US-02 Browse
-  US-03 View book
-  US-04 Place order
-  US-05 Cancel order
-  US-06 Alerts
-  US-07 Add book
-  US-08 Update profile

- Each story includes acceptance criteria & DoD
- TbD with the Development Team : Story Point

Technical Focus

Security:

- API keys
- Bcrypt hashing
- HTTPS mandatory, rate limit
- GDPR compliant (90d)

Performance:

- Optimized DB Indexes
- Background & async jobs (notifications)
- Responses < 200 ms (GET)

Monitoring:

- Audit logs & error tracking.

Next Steps & Conclusion

1. Implement JWT authentication
2. Complete the end-to-end orders workflow
3. Automate notification background job

Questions?